JavaScript is disabled on your browser.

compact1, compact2, compact3

java.time.zone

Class ZoneRules

java.lang.Object
    java.time.zone.ZoneRules

All Implemented Interfaces:

Serializable

```
public final class ZoneRules
extends Object
implements Serializable
```

The rules defining how the zone offset varies for a single time-zone.

The rules model all the historic and future transitions for a time-zone. `ZoneOffsetTransition` is used for known transitions, typically historic. `ZoneOffsetTransitionRule` is used for future transitions that are based on the result of an algorithm.

The rules are loaded via `ZoneRulesProvider` using a `ZoneId`. The same rules may be shared internally between multiple zone IDs.

Serializing an instance of `ZoneRules` will store the entire set of rules. It does not store the zone ID as it is not part of the state of this object.

A rule implementation may or may not store full information about historic and future transitions, and the information stored is only as accurate as that supplied to the implementation by the rules provider. Applications should treat the data provided as representing the best information available to the implementation of this rule.

Implementation Requirements:

`This class is immutable and thread-safe.`

Since:

1.8

See Also:

Serialized Form

Method Summary

Concrete Methods

Modifier and Type	Method
boolean	**equals**(**Object** otherRules)
Duration	**getDaylightSavings**(**Instant** instant)
ZoneOffset	**getOffset**(**Instant** instant)
ZoneOffset	**getOffset**(**LocalDateTime** localDateTime)
ZoneOffset	**getStandardOffset**(**Instant** instant)

ZoneOffsetTransition	**getTransition**(**LocalDateTime** localDateTime)
List<**ZoneOffsetTransitionRule**>	**getTransitionRules**()
List<**ZoneOffsetTransition**>	**getTransitions**()
List<**ZoneOffset**>	**getValidOffsets**(**LocalDateTime** localDateTime)
int	**hashCode**()

boolean	**isDaylightSavings**(**Instant** instant)
boolean	**isFixedOffset**()
boolean	**isValidOffset**(**LocalDateTime** localDateTime, **ZoneOffset** offset)
ZoneOffsetTransition	**nextTransition**(**Instant** instant)
static **ZoneRules**	**of**(**ZoneOffset** offset)
static **ZoneRules**	**of**(**ZoneOffset** baseStandardOffset, **ZoneOffset** baseWallOffset, **List**<**ZoneOffsetTransition**> standardOffsetTransiti **List**<**ZoneOffsetTransition**> transitionList, **List**<**ZoneOffsetTransitionRule**> lastRules)
ZoneOffsetTransition	**previousTransition**(**Instant** instant)

String	toString()

Methods inherited from class java.lang.Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Method Detail

of

```
public static ZoneRules of(ZoneOffset baseStandardOffset,
                           ZoneOffset baseWallOffset,
                           List<ZoneOffsetTransition> standardOffsetTransitionLis
                           List<ZoneOffsetTransition> transitionList,
                           List<ZoneOffsetTransitionRule> lastRules)
```

Obtains an instance of a ZoneRules.

Parameters:

baseStandardOffset - the standard offset to use before legal rules were set, not null

baseWallOffset - the wall offset to use before legal rules were set, not null

standardOffsetTransitionList - the list of changes to the standard offset, not null

transitionList - the list of transitions, not null

lastRules - the recurring last rules, size 16 or less, not null

Returns:

the zone rules, not null

of

```
public static ZoneRules of(ZoneOffset offset)
```

Obtains an instance of ZoneRules that has fixed zone rules.

Parameters:

offset - the offset this fixed zone rules is based on, not null

Returns:

the zone rules, not null

See Also:

isFixedOffset()

isFixedOffset

```
public boolean isFixedOffset()
```

Checks of the zone rules are fixed, such that the offset never varies.

Returns:

true if the time-zone is fixed and the offset never changes

getOffset

```
public ZoneOffset getOffset(Instant instant)
```

Gets the offset applicable at the specified instant in these rules.

The mapping from an instant to an offset is simple, there is only one valid offset for each instant. This method returns that offset.

Parameters:

instant - the instant to find the offset for, not null, but null may be ignored if the rules have a single offset for all instants

Returns:

the offset, not null

getOffset

```
public ZoneOffset getOffset(LocalDateTime localDateTime)
```

Gets a suitable offset for the specified local date-time in these rules.

The mapping from a local date-time to an offset is not straightforward. There are three cases:

- Normal, with one valid offset. For the vast majority of the year, the normal case applies, where there is a single valid offset for the local date-time.
- Gap, with zero valid offsets. This is when clocks jump forward typically due to the spring daylight savings change from "winter" to "summer". In a gap there are local date-time values with no valid offset.

- Overlap, with two valid offsets. This is when clocks are set back typically due to the autumn daylight savings change from "summer" to "winter". In an overlap there are local date-time values with two valid offsets.

Thus, for any given local date-time there can be zero, one or two valid offsets. This method returns the single offset in the Normal case, and in the Gap or Overlap case it returns the offset before the transition.

Since, in the case of Gap and Overlap, the offset returned is a "best" value, rather than the "correct" value, it should be treated with care. Applications that care about the correct offset should use a combination of this method, `getValidOffsets(LocalDateTime)` and `getTransition(LocalDateTime)`.

Parameters:

`localDateTime - the local date-time to query, not null, but null may be ignored if the rules have a single offset for all instants`

Returns:

`the best available offset for the local date-time, not null`

getValidOffsets

```
public List<ZoneOffset> getValidOffsets(LocalDateTime localDateTime)
```

Gets the offset applicable at the specified local date-time in these rules.

The mapping from a local date-time to an offset is not straightforward. There are three cases:

- Normal, with one valid offset. For the vast majority of the year, the normal case applies, where there is a single valid offset for the local date-time.
- Gap, with zero valid offsets. This is when clocks jump forward typically due to the spring daylight savings change from "winter" to "summer". In a gap there are local date-time values with no valid offset.
- Overlap, with two valid offsets. This is when clocks are set back typically due to the autumn daylight savings change from "summer" to "winter". In an overlap there are local date-time values with two valid offsets.

Thus, for any given local date-time there can be zero, one or two valid offsets. This method returns that list of valid offsets, which is a list of size 0, 1 or 2. In the case where there are two offsets, the earlier offset is returned at index 0 and the later offset at index 1.

There are various ways to handle the conversion from a `LocalDateTime`. One technique, using this method, would be:

```
  List<ZoneOffset> validOffsets = rules.getOffset(localDT);
  if (validOffsets.size() == 1) {
    // Normal case: only one valid offset
```

```
    zoneOffset = validOffsets.get(0);
  } else {
    // Gap or Overlap: determine what to do from transition (which will be non-nu
    ZoneOffsetTransition trans = rules.getTransition(localDT);
  }
```

In theory, it is possible for there to be more than two valid offsets. This would happen if clocks to be put back more than once in quick succession. This has never happened in the history of time-zones and thus has no special handling. However, if it were to happen, then the list would return more than 2 entries.

Parameters:

`localDateTime - the local date-time to query for valid offsets, not null, but null may be ignored if the rules have a single offset for all instants`

Returns:

`the list of valid offsets, may be immutable, not null`

getTransition

`public ZoneOffsetTransition getTransition(LocalDateTime localDateTime)`

Gets the offset transition applicable at the specified local date-time in these rules.

The mapping from a local date-time to an offset is not straightforward. There are three cases:

- Normal, with one valid offset. For the vast majority of the year, the normal case applies, where there is a single valid offset for the local date-time.
- Gap, with zero valid offsets. This is when clocks jump forward typically due to the spring daylight savings change from "winter" to "summer". In a gap there are local date-time values with no valid offset.
- Overlap, with two valid offsets. This is when clocks are set back typically due to the autumn daylight savings change from "summer" to "winter". In an overlap there are local date-time values with two valid offsets.

A transition is used to model the cases of a Gap or Overlap. The Normal case will return null.

There are various ways to handle the conversion from a `LocalDateTime`. One technique, using this method, would be:

```
  ZoneOffsetTransition trans = rules.getTransition(localDT);
  if (trans == null) {
    // Gap or Overlap: determine what to do from transition
```

```
  } else {
    // Normal case: only one valid offset
    zoneOffset = rule.getOffset(localDT);
  }
```

Parameters:

`localDateTime - the local date-time to query for offset transition, not null, but null may be ignored if the rules have a single offset for all instants`

Returns:

`the offset transition, null if the local date-time is not in transition`

getStandardOffset

```
public ZoneOffset getStandardOffset(Instant instant)
```

Gets the standard offset for the specified instant in this zone.

This provides access to historic information on how the standard offset has changed over time. The standard offset is the offset before any daylight saving time is applied. This is typically the offset applicable during winter.

Parameters:

`instant - the instant to find the offset information for, not null, but null may be ignored if the rules have a single offset for all instants`

Returns:

`the standard offset, not null`

getDaylightSavings

```
public Duration getDaylightSavings(Instant instant)
```

Gets the amount of daylight savings in use for the specified instant in this zone.

This provides access to historic information on how the amount of daylight savings has changed over time. This is the difference between the standard offset and the actual offset. Typically the amount is zero during winter and one hour during summer. Time-zones are second-based, so the nanosecond part of the duration will be zero.

This default implementation calculates the duration from the `actual` and `standard` offsets.

Parameters:

`instant - the instant to find the daylight savings for, not null, but null may be ignored if the rules have a single offset for all instants`

Returns:

`the difference between the standard and actual offset, not null`

isDaylightSavings

```
public boolean isDaylightSavings(Instant instant)
```

Checks if the specified instant is in daylight savings.

This checks if the standard offset and the actual offset are the same for the specified instant. If they are not, it is assumed that daylight savings is in operation.

This default implementation compares the `actual` and `standard` offsets.

Parameters:

`instant - the instant to find the offset information for, not null, but null may be ignored if the rules have a single offset for all instants`

Returns:

`the standard offset, not null`

isValidOffset

```
public boolean isValidOffset(LocalDateTime localDateTime,
                             ZoneOffset offset)
```

Checks if the offset date-time is valid for these rules.

To be valid, the local date-time must not be in a gap and the offset must match one of the valid offsets.

This default implementation checks if `getValidOffsets(java.time.LocalDateTime)` contains the specified offset.

Parameters:

`localDateTime - the date-time to check, not null, but null may be ignored if the rules have a single offset for all instants`

`offset - the offset to check, null returns false`

Returns:

`true if the offset date-time is valid for these rules`

nextTransition

```
public ZoneOffsetTransition nextTransition(Instant instant)
```

Gets the next transition after the specified instant.

This returns details of the next transition after the specified instant. For example, if the instant represents a point where "Summer" daylight savings time applies, then the method will return the transition to the next "Winter" time.

Parameters:

`instant - the instant to get the next transition after, not null, but null may be ignored if the rules have a single offset for all instants`

Returns:

`the next transition after the specified instant, null if this is after the last transition`

previousTransition

```
public ZoneOffsetTransition previousTransition(Instant instant)
```

Gets the previous transition before the specified instant.

This returns details of the previous transition after the specified instant. For example, if the instant represents a point where "summer" daylight saving time applies, then the method will return the transition from the previous "winter" time.

Parameters:

`instant - the instant to get the previous transition after, not null, but null may be ignored if the rules have a single offset for all instants`

Returns:

`the previous transition after the specified instant, null if this is before the first transition`

getTransitions

```
public List<ZoneOffsetTransition> getTransitions()
```

Gets the complete list of fully defined transitions.

The complete set of transitions for this rules instance is defined by this method and `getTransitionRules()`. This method returns those transitions

that have been fully defined. These are typically historical, but may be in the future.

The list will be empty for fixed offset rules and for any time-zone where there has only ever been a single offset. The list will also be empty if the transition rules are unknown.

Returns:

`an immutable list of fully defined transitions, not null`

getTransitionRules

`public List<ZoneOffsetTransitionRule> getTransitionRules()`

Gets the list of transition rules for years beyond those defined in the transition list.

The complete set of transitions for this rules instance is defined by this method and `getTransitions()`. This method returns instances of `ZoneOffsetTransitionRule` that define an algorithm for when transitions will occur.

For any given `ZoneRules`, this list contains the transition rules for years beyond those years that have been fully defined. These rules typically refer to future daylight saving time rule changes.

If the zone defines daylight savings into the future, then the list will normally be of size two and hold information about entering and exiting daylight savings. If the zone does not have daylight savings, or information about future changes is uncertain, then the list will be empty.

The list will be empty for fixed offset rules and for any time-zone where there is no daylight saving time. The list will also be empty if the transition rules are unknown.

Returns:

`an immutable list of transition rules, not null`

equals

`public boolean equals(Object otherRules)`

Checks if this set of rules equals another.

Two rule sets are equal if they will always result in the same output for any given input instant or local date-time. Rules from two different groups may return false even if they are in fact the same.

This definition should result in implementations comparing their entire state.

Overrides:

`equals` in class `Object`

Parameters:

`otherRules - the other rules, null returns false`

Returns:

`true if this rules is the same as that specified`

See Also:

`Object.hashCode()`, `HashMap`

hashCode

```
public int hashCode()
```

Returns a suitable hash code given the definition of `#equals`.

Overrides:

`hashCode` in class `Object`

Returns:

`the hash code`

See Also:

`Object.equals(java.lang.Object)`, `System.identityHashCode(java.lang.Object)`

toString

```
public String toString()
```

Returns a string describing this object.

Overrides:

`toString` in class `Object`

Returns:

```
a string for debugging, not null
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.